3KIDNAS Fortran Issues Found During DCP/JS Port

Prepared 2026-07-24 while porting the InitialAnalysis pre-fit chain (moment maps, shape/radial-profile estimation, noise) to JS for the DCP bootstrap pipeline. All four issues below were found by reading the Fortran source directly against what a faithful line-for-line port would need to reproduce — not by guessing or by observing wrong output first. None have been fixed in Fortran; each was reproduced *faithfully* in the JS port (matching current behavior exactly) rather than silently corrected, so the two implementations stay comparable until you've had a chance to weigh in on each one.

Contents

1 Moment maps silently drop the last channel

`src/MomentMaps/CalculateMomentMaps.f — MakeMomentMaps (line 16), CalculateGeneralMoment (line 65)`

MEDIUM — affects every moment-0/1/2 map, every InitialAnalysis run (initial fit and every bootstrap realization).

What happens: `MakeMomentMaps` calls `CalculateGeneralMoment` passing `n = DC%DH%nChannels - 1` as the array-length argument:

```
call CalculateGeneralMoment(DC%DH%nChannels-1
     &          ,DC%Channels(0:DC%DH%nChannels-1)
     &          ,DC%Flux(i,j,0:DC%DH%nChannels-1)
     &          ,Moment,Maps%Flux(i,j,Moment))
```

Inside `CalculateGeneralMoment`, the dummy arrays are declared using that same `n`:

```
subroutine CalculateGeneralMoment(n,Arr,WArr,WMom
     &                  ,CalcMoment)
      ...
      real, INTENT(IN) :: Arr(0:n-1), WArr(0:n-1)
      ...
      do i=0,n-1
          ...
      enddo
```

Because `n = nChannels - 1`, the dummy arrays are declared with only `nChannels - 1` elements (indices `0` to `nChannels-2`), and the summation loop only ever visits those indices. **The last channel (index `nChannels-1`) is never included** in moment 0 (total flux), moment 1 (mean velocity), or moment 2 (dispersion), for any pixel, on any run.

> **Likely intent:** passing `nChannels` (not `nChannels-1`) as the length argument, matching the full range actually loaded into `Arr`/`WArr` at the call site (`0:DC%DH%nChannels-1`, which *is* the full channel range — only the length argument passed separately is one short). This looks like an off-by-one introduced when translating a 0-based "last valid index" convention into the "number of elements" argument `CalculateGeneralMoment` actually expects.

Impact: systematic, silent truncation of every moment map used for center/shape/radial-profile estimation. The effect on a typical cube (tens of channels) is small per-pixel but is a real, fixed bias present in *every* fit, not just bootstrap realizations.

2 `FillInMisingRings` unconditionally aborts after successfully filling ring 0

`src/PreAnalysis/EstimateRadialProfiles.f — FillInMisingRings, lines 534–581 (the relevant branch: 548–565)`

HIGH — can stop the program even when a valid fallback value was found.

What happens: when ring 0's surface density is not modelable, the routine searches rightward for a fallback ring and, if found, copies its values in — then **unconditionally prints an error and calls `stop` anyway**, regardless of whether that search succeeded:

```
if(ModelerableRingSwitch(i) .eq. 0) then
    if(i .eq. 0) then
        do k=i, nRings-1
            if(ModelerableRingSwitch(k) .eq. 1) then
                print*, "Fill in ring by", i,k
     &              ,RadialProfile(0,i)
     &              ,RadialProfile(1:2,k)
                RadialProfile(1:2,i)=RadialProfile(1:2,k)
                exit
            endif
        enddo
c       If there is no ring with a viable value is found, then there is no point in running t
        print*, "No ring with an acceptable"
     &          //" surface density found"
        stop
    else
        ...
```

There is no `if`/guard between the end of the `do k=...` loop and the `print`/`stop` — they execute every time `i==0` takes this branch, whether or not `exit` fired inside the loop. The comment directly above the `stop` ("If there is no ring with a viable value is found...") states the intended condition, but nothing in the code actually checks it.

> **Likely intent:** the `stop` should only fire if the `do k` loop completes *without* finding a modelable ring (e.g. guarded by a found-flag checked after the loop) — matching what the surrounding comments describe. As written, ring 0 being non-modelable always terminates the program, even in the common case where a perfectly good fallback ring exists immediately to its right.

Impact: potentially the most consequential of the four — this is a hard program abort (in the local path) or slice failure (once ported), not a numerical discrepancy. Whether it's been observed in practice likely depends on how often ring 0's surface density fails modelability for real data (the innermost ring is often noisier due to beam smearing, so this isn't a purely theoretical edge case).

3 `GetGalaxyShape`'s configurable shape-source branch is unreachable

`src/PreAnalysis/InitialAnalysis.f — GetGalaxyShape, lines 183–218` **CODE QUALITY** — not a numerical bug, but worth flagging: `PFlags%ShapeSource` has no effect.

What happens: the routine branches on `PFlags%ShapeSource` to compute `Incl`/`PA` either via `EstimateShape` (moment-based) or a catalogue ellipse-axis-ratio formula:

```
if(PFlags%ShapeSource .eq. 0) then
    call EstimateShape(ObservedMaps,Center
     &                      ,Incl,PA)
elseif(PFlags%ShapeSource .eq. 1) then
    PA=CatItem%kinPA*Pi/180.+Pi/2.
    if(PA .gt. 2.*Pi) PA=PA-2.*Pi
    Ellip=CatItem%EllipseMin/CatItem%EllipseMaj
    Incl=acos(ellip)
endif
...
Incl=CatItem%EllipseInc*Pi/180.
PA=(CatItem%EllipsePA+90.)*Pi/180.
if(PA .lt. 0.) then
    PA=PA+2.*Pi
endif
if(PA .gt. 2.*Pi) then
    PA=PA-2.*Pi
endif
```

Immediately after the `if`/`elseif` block, both `Incl` and `PA` are **unconditionally overwritten** from the SoFiA catalogue's own `EllipseInc`/`EllipsePA` fields — with no branch that skips this override. Whatever `ShapeSource` is set to, and whatever the `if`/`elseif` block computed, is discarded before `GetGalaxyShape` returns.

> **Note:** this one may be entirely intentional — e.g. a deliberate "always trust the SoFiA catalogue" decision made after the config-driven paths were written, with the dead branch simply left in place. Flagging it because it means `PFlags%ShapeSource` is currently a no-op, which may be surprising if anyone is relying on it to switch behavior.

Impact: none on current numerical output (the JS port reproduces the actually-executed catalogue-based path only). Purely a "is this config flag supposed to do something" question.

4 All-rings-modelable case can index outside the allocated profile array

`src/PreAnalysis/EstimateRadialProfiles.f — EstimateProfiles, lines 89 (allocation) and 124–126 (reassignment)`

MEDIUM — edge case, likely rare with real (noisy) data; surfaced while testing this port against small synthetic cubes.

What happens: the wide profile array is allocated using the geometry-derived ring count (`nRingsMax`):

```
ALLOCATE(EstimatedProfile(0:2,-nRings+1:nRings))
```

Later, in automatic mode (`FittingOptions%nTargRings == -1`), `nRings` is reassigned to include one deliberate extra "trial" ring:

```
if(FittingOptions%nTargRings .eq. -1) then
    nRings=sum(ModelerableRingSwitch)+1 !Add an extra ring trial
else
    nRings=FittingOptions%nTargRings
endif
```

`ModelerableRingSwitch` has `nRingsMax` entries. If *every* candidate ring turns out modelable, `sum(ModelerableRingSwitch) == nRingsMax`, so the reassigned `nRings` becomes `nRingsMax + 1` — one more than `EstimatedProfile`'s allocated range. The leading/trailing velocity-profile loops later in the same subroutine then read `EstimatedProfile` at an index one step outside its declared bounds.

> **How this was found:** while integration-testing the JS port against a small, clean synthetic disc, every ring came back modelable and the equivalent JS code threw on an out-of-bounds array read at exactly this point. Retested with a synthetic cube shaped so a few outer rings fail modelability (much closer to how real, noisier data behaves) and the rest of the pipeline ran correctly — so this specific accounting issue, not a broader problem, is what's flagged here.

Likely intent: the "+1 extra ring trial" logic doesn't appear to account for the case where there's no non-modelable ring left to trial beyond. Possibly needs a min() against `nRingsMax`, or the allocation should reserve one extra slot up front.

Impact: unconfirmed against a real Fortran run with real galaxy data (where 100% ring modelability may simply be uncommon enough that this never triggers in practice) — flagging as a real, reachable code path rather than a confirmed production incident.

Found while porting `src/PreAnalysis/*.f` to JS for the DCP-distributed bootstrap pipeline (third_party/DCP/). Every finding above was verified by reading the Fortran source directly (line numbers cited above match this repository's current state as of 2026-07-24) rather than inferred from behavior. Happy to walk through any of these in more detail, or help instrument a targeted Fortran-side check to confirm behavior against real data before deciding whether/how to fix each one.